

龙芯杯初赛设计报告

借阅声明

1. 本次龙芯杯团体赛我们参考了 Open LA500 和开源 RISC-V 风格的处理器 biriscv (<https://github.com/ultraembedded/biriscv>), 遵循 Apache-2.0 许可证, 并严格将对应的 license 版权声明放在了本项目部分 rtl 代码中。本项目设计并实现了双发射流水线、分支预测、LoongArch32 指令、AXI 接口、双 Cache 时序、TLB 设计、精确中断异常、乘除法器、Chiplab 仿真测试、FPGA 适配及调试监测等改造与集成, 通过了所有的功能、性能测试点同时借助 U-Boot 启动了 Linux 操作系统并成功运行了 ls 指令。在性能测试主频设定为 80MHz 的情形下 CoreMark 的仿真性能得分可达 3.41, 实际下板测得性能分为 3.20, IPC 可达 1.20, 计数器加速比为 2.84。
2. 乘法器的设计我们采用了组员王丰源同学改进后的个人软件著作权的 32 位乘法自动生成软件生成。但声明一点: 由于我们自行设计的乘法器采用 LUT 触发器 + 流水寄存器的形式, 对于基 4-booth 算法 + wallace 树的庞大乘法器, 在仿真 CoreMark 跑分时会因为多周期的引入从而使仿真 CoreMark 性能分骤降至 2.83, 因此经过反复斟酌我们决定初赛提交的作品里采用宏注释的方法没有将我们自行设计的乘法器综合入 CPU 中, 但后续决赛的 Linux 系统启动环节我们已经实现了基于自行设计的乘法 ip 成功启动了操作系统。整个乘法器的所有关键添加路径均由一个 ``define CPU_REAL_MDU` 宏区分开来, 只需开启该宏就能将所有的乘法逻辑切换至我们的乘法 ip。
3. 由于性能跑分相关的除法指令并不算多, 除法指令依旧沿用了组员王丰源去年参赛使用的除法器 ip, 但是限定了除法指令识别后 CPU 执行期间改为单发射 slot0 提交, 因为旧的除法 ip 的执行周期数为 32 个周期。

一、设计总述

我们设计的是一个 32-bit LoongArch 风格的顺序双发射 CPU：一次取 64-bit 两条指令，前端用 BTB/BHT/RAS 预测下一 PC 值，issue 用 4 项 packet queue、scoreboard 和多级旁路决定 slot0/slot1，后端以两条 E1/E2/WB pipe 顺序提交；I-cache 与 D-cache 各 16KiB、2 路、32B Cache line，经 AXI arbiter 合并为一套外部总线 AXI master。CPU 的总体设计如图 1 所示。

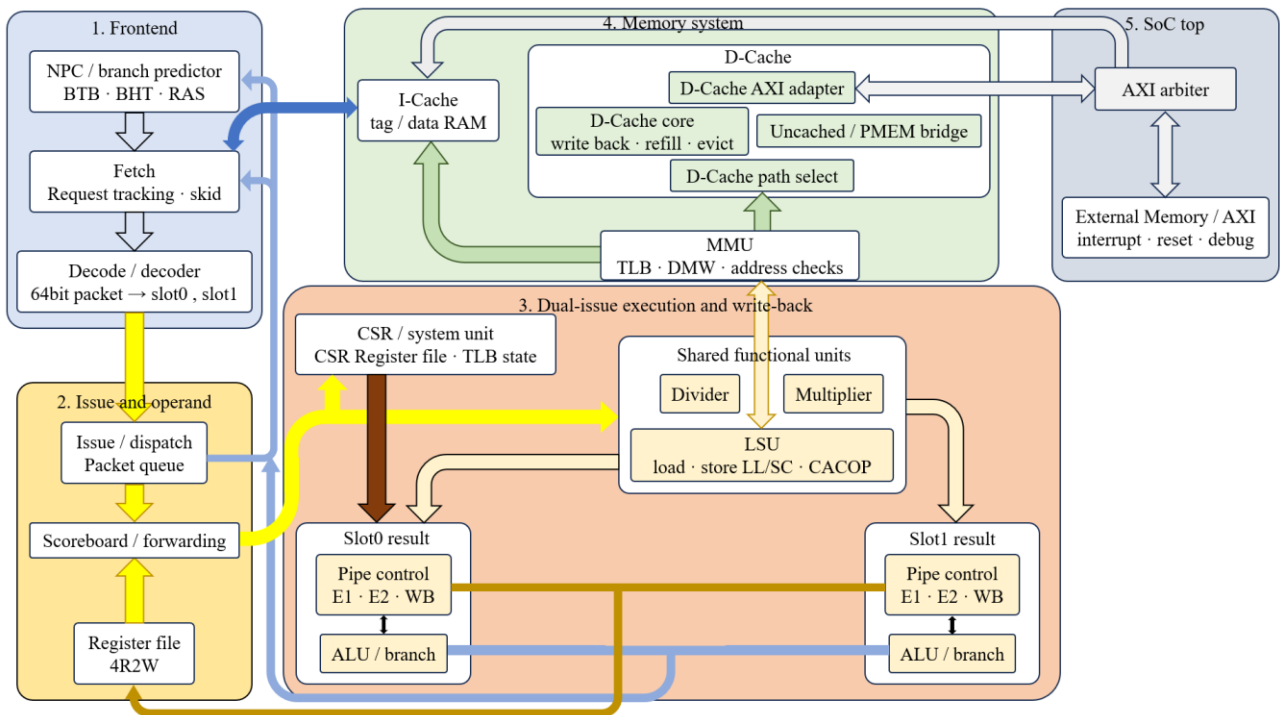


图 1 项目总体结构图

二、总体设计方案

本处理器以顺序双发射为总体组织方式。所谓“顺序”，是指指令仍按程序规定的先后关系完成，便于保证异常和访存行为正确；所谓“双发射”，是指在两条指令彼此独立、执行资源允许时，同一周期可将它们分别送入 slot0 和 slot1 两条执行通路。这样的设计在保持控制逻辑可验证性的同时，提高了单位周期内能够完成的有效工作量。

从顶层看，`core_top.v` 负责连接 Chiplab 平台的复位、外部中断、调试信息和 AXI 接口；`biloong_top.v` 将处理器核心、I-cache 和 D-cache 组合起来；`biloong_core.v` 内部再连接前端、issue、执行单元、LSU、CSR 和 MMU。因此可以把整个系统理解为“前端准备指令，issue 安排指令，执行单元完成计算，Cache 与 AXI 提供存储访问”四个部分。

（一）流水线与双发射设计

1. 指令在处理器中的流动

前端一次从 I-cache 取得 64-bit 指令包，其中包含相邻的两条 32-bit 指令，分别称为 lower slot 和 upper slot。经过 fetch 与 decode 后，指令包进入容量为 4 项的 packet queue。issue 模块从队首观察两条指令是否存在数据依赖、功能单元冲突或异常顺序限制，再决定本周期发射一条还是两条。

NPC 选择下一 PC → I-cache 取指 → decode 拆分两条指令 → issue 判断依赖 → E1/E2 执行 → WB 写回

源码中的 scoreboard 可以通俗地理解为“寄存器占用表”：如果一条较早指令还没有产生某个寄存器的结果，较晚指令就不能直接读取旧值。若结果已经由执行单元产生但尚未写回，forwarding（旁路）会把结果直接送给下一条指令，减少等待 Regfile 写回再读出的停顿。

slot0 和 slot1 并非完全对称。slot0 可承担更完整的系统控制和异常处理任务，某些长延迟或有严格顺序要求的操作只允许从 slot0 发射；普通算术、分支、乘法和部分访存操作在条件满足时可以使用 slot1。无论单发射还是双发射，最终都由两条 pipe_ctrl 通路按照程序顺序进入 WB，从而避免“后面的指令先提交”带来的状态错误。

（二）分支预测模块设计

1. 为什么需要分支预测

普通指令通常按 PC、PC+4、PC+8 的顺序执行，但条件分支、函数调用和函数返回会改变下一条指令地址。真正的分支结果要到执行阶段才能确定，如果前端每次都停下来等待，流水线会出现明显空泡。分支预测单元位于 biloong_npc.v，它根据过去执行过的控制流提前给出 next_pc_f_o，使 I-cache 可以继续取指。预测正确时不增加额外停顿；预测错误时，处理器丢弃错误路径上的年轻指令并从真实地址重新取指，因此预测只影响性能，不影响程序正确性。

2. BTB、BHT 与 RAS 的分工

表 1 分支预测说明

结构	规模	作用	源码名
BTB	32 项、2 路	记住“哪条指令发生过跳转”以及它通常跳到哪里，相当于控制流地址簿。	btb_pc_way*_q、 btb_target_way*_q
BHT	512 项、每 项 2 bit	记录条件分支更偏向跳转还是顺序执行；连续结果会逐步增强或减弱预测倾向。	bht_sat_q、global_history_q
RAS	8 项	专门保存函数调用后的返回地址，使 ret 指令能够快速回到调用点。	ras_stack_q、ras_index_q

如表 1 所示，BTB（Branch Target Buffer）首先判断当前 PC 是否曾经是一条分支或跳转指令，并给出候选目标地址；BHT（Branch History Table）主要判断条件分支本次更可能“跳”还是“不跳”；RAS（Return Address Stack）则专门处理函数返回。三者并不是重复工作，而是分别回答“它是不是控制流指令”、“条件是否可能成立”、“函数应返回到哪里”。

BHT 中的 bht_sat_q 使用 2-bit 饱和计数器。计数值较大时预测 taken，较小时预测 not taken；真实结果由后端确认后，计数器只向正确方向移动一级，不会因为一次偶然结果就立刻完全翻转。global_history_q 还保存最近若干次分支的整体走向，并与当前 PC 一起选择 BHT 项，使循环和具有上下文规律的判断更容易被识别。

RAS 中同时维护预测使用的 ras_index_q 和后端确认的 ras_index_real_q。前端预测到 call 或 ret 时可以提前压栈或弹栈；如果后来发现预测路径错误，则以真实执行状态重新校准预测栈指针。这样既保留了提前取指的速度，也避免错误路径长期污染返回地址。

3. 双发射条件下的预测

由于一次取指包含 lower 和 upper 两条指令，预测器会同时检查 pc_f_i 与 pc_f_i+4 对应的位置，并用 next_taken_f_o[1:0]标明哪个 slot 被预测为跳转。若 lower slot 已经预测跳转，upper slot 就不应继续执行，因此 lower 的优先级高于 upper。这一规则保证预测结果与程序顺序一致，也避免一包内出现两个相互矛盾的跳转目标。

4. 预测、核对与纠错流程

pc_f_i 查询 BTB/BHT/RAS→产生 next_pc_f_o→fetch 接受预测→执行阶段得到真实结果→更新或重定向

当前取指地址由 pc_f_i 输入。预测器综合 BTB 命中、BHT 方向和 RAS 返回地址后产生

next_pc_f_o; 只有 pc_accept_i 有效, 才表示前端真正采用了这次预测。分支指令进入执行阶段后, branch_request_i 表示真实结果已经产生, branch_source_i 给出分支指令地址, branch_pc_i 给出真实目标地址, branch_is_taken_i 与 branch_is_not_taken_i 给出实际方向。

若预测方向和目标均正确, 流水线继续运行, 同时更新相应预测信息; 若预测错误, issue 与前端发出 redirect, 清除错误路径上的年轻指令, 并把 NPC 改到 branch_pc_i 或正确的顺序地址。设计中还保留 BPU_PERF 相关统计, 可分别观察预测次数、命中和误预测, 为后续调整 BTB/BHT 参数提供依据。

(三) Cache 子系统设计

1. 总体结构与参数

处理器将一级缓存分为 I-cache 和 D-cache。I-cache 只服务取指, D-cache 服务 load/store 与 Cache 维护操作。分离结构使取指和数据访问在大多数情况下可以并行进行, 不必争用同一个内部存储端口; 两者最终再通过 axi_arbiter.v 合并到一套外部 AXI master。

表 2 Cache 设计说明

项目	I-cache	D-cache
容量与组织	16 KiB、2 路、256 组	16 KiB、2 路、256 组
Cache line	32B, 每次 AXI 回填 8 个 32-bit beat	32B, 每次 AXI 回填或写回 8 个 32-bit beat
处理对象	64-bit 取指包, 即两条 32-bit 指令	load/store、CACOP 和非缓存访问
写策略	只读, 不产生 dirty 数据	write-back、write-allocate

如表 2 所示, “2 路、256 组、32B line”可以理解为 256 个编号抽屉, 每个抽屉有两个位置, 每个位置保存连续 32 字节。访问地址被拆成 tag、set index 和 line offset: set index 决定查看哪个抽屉, tag 判断抽屉中的内容是否属于当前地址, line offset 决定取这一行中的哪个字或指令包。两路结构允许同一组中保存两个不同地址块, 降低直接映射造成的频繁互相替换。

2. I-cache 的命中与回填

前端用 req_rd_i 发起正式取指请求, I-cache 用 req_accept_o 表示已经接收; 若 tag 匹配, 则 req_valid_o 拉高, 并通过 req_inst_o[63:0]返回两条指令。accept 与 valid 的区别是: 前者表

示“请求收到了”，后者表示“结果已经可以使用”。这种区分允许 Cache 在 miss 时先保存请求，再经过若干周期返回数据。

I-cache 主要包含 STATE_FLUSH、*_LOOKUP、*_REFILL 以及 *_RELOOKUP 四个状态。复位后先在 STATE_FLUSH 中逐组清除有效位；正常取指处于 STATE_LOOKUP；未命中时进入 STATE_REFILL，从 AXI 读取对齐后的整条 32B cache line；回填完成后进入 STATE_RELOOKUP，让同步 RAM 重新建立原请求的读出数据，再回到 LOOKUP 返回指令。

STATE_LOOKUP：命中直接返回；未命中→STATE_REFILL：接收 8 个 beat→STATE_RELOOKUP：重新查询→返回指令

req_lookup_pc_i 与 req_pc_i 是 I-cache 的一个时序优化。前者使用地址中不受翻译影响的部分提前启动 RAM 查询，后者使用 MMU 给出的最终物理地址完成 tag 比较和 AXI 回填。两条路径并行进行，避免“先完成地址翻译、再开始读取 Cache RAM”的长组合路径。

3. D-cache 的读写流程

LSU 通过 mem_rd_i、mem_wr_i 和地址信息向 D-cache 发起 load/store。mem_accept_o 表示请求已被接收，mem_ack_o 表示该请求真正完成；mem_req_tag_i 用于把返回结果与 LSU 中保存的请求对应起来。biloong_top.v 在 LSU 与 D-cache 之间还设置了一项 skid buffer，当 Cache 暂时忙碌时先保存完整请求，避免地址、写数据或控制信号在反压期间丢失。

load 命中时，D-cache 从命中的 way 读取 32-bit 对齐数据并返回 LSU；store 命中时，根据 mem_wr_i[3:0]只修改目标字节，同时把该行的 dirty 位置为 1。dirty 表示 Cache 中的数据已比主存更新。当前 D-cache 采用 write-back：写命中时不立即访问 AXI，只有该行被替换、显式 writeback 或执行全 Cache flush 时才写回主存。

store 未命中时采用 write-allocate。Cache 先把目标 32B 行从内存读入，再在 STATE_WRITE 中修改需要写入的字节。这样做会增加本次 miss 的开销，但后续访问同一行更容易命中，适合具有局部性的普通程序。

4. D-cache miss、替换与写回

D-cache 在 STATE_LOOKUP 中比较两个 way 的 valid 与 tag。若未命中，先选择一个替换 way；若旧行无效或干净，可直接进入 STATE_REFILL；若旧行 valid 且 dirty，则先经过 STATE_EVICT_PREP、STATE_EVICT 和 STATE_EVICT_WAIT，把旧行的 8 个 32-bit word 送到 AXI 并等待写响应，然后再回填新行。

LOOKUP miss→dirty?→EVICT_PREP/EVICT/EVICT_WAIT→REFILL→READ 或 WRITE→LOOKUP 完成

表 3 D-Cache 状态机设计

状态组	作用	结果
LOOKUP	判断命中、接收普通请求和 Cache 维护命令	命中时快速产生 mem_ack_o
REFILL/READ/WRITE	从 AXI 回填整行，并完成原 load 或 store	miss 请求在回填后完成
EVICT_PRE/EVICT/ EVICT_WAIT	把被替换的 dirty 行写回并等待响应	保护主存中的最新数据
INVALIDATE/ WRITEBACK/FLUSH	处理单行无效、单行写回和全 Cache 清理	对应 CACOP 或系统维护操作

如表 3 所示，EVICT_PREP 的存在与 FPGA 同步 RAM 时序有关：状态机先给出被替换行第一个 word 的地址，下一拍数据才稳定，再连续发送后续 word。写回地址由旧 tag、当前 set index 和行内全零 offset 重新拼接，确保写回的是被替换的旧行，而不是新请求地址。dirty 位要等 AXI 写响应确认后才能清除，不能在最后一个写数据 beat 发出时提前清除。

5. cached、uncached 与 Cache 维护

mem_cacheable_i 决定访问是否进入真正的 D-cache。普通可缓存内存进入 dcache_core；串口、CONFREG 等 MMIO 或被标记为非缓存的地址进入 dcache_if_pmem，直接形成单 beat AXI 访问。这样既能让普通内存利用局部性，也能保证设备寄存器每次都真正到达外设，不会被 Cache 命中掩盖。

dcache_mux.v 会记录当前未完成事务属于 cached 还是 uncached 路径；若旧事务尚未返回，新请求却要切换路径，hold_w 会暂时阻止新请求进入，避免返回数据被送到错误一侧。对软件可见的 Cache 维护则由 invalidate、writeback 和 flush 完成：invalidate 丢弃指定行，writeback 把指定 dirty 行写回，flush 遍历全部组并写回需要保存的数据。

（四）AXI 仲裁与访存可靠性

I-cache 只使用 AXI 的 AR/R 读通道，D-cache 同时使用 AR/R 与 AW/W/B 读写通道。axi_arbiter.v 将两套内部接口合并为单一外部 AXI master。两侧同时申请读取时优先服务 D-cache，以缩短 load/store 对流水线的直接阻塞；仲裁器一次只保留一个外部读 burst 在途，并由 rd_owner_q 记录该请求来自 I-cache 还是 D-cache，返回的 R 数据据此送回正确模块。

AXI 的 ready 可能随时拉低，因此 AR、AW、W 的地址或数据在握手完成前必须保持稳定。设计中的 ar_hold_*、aw_hold_* 和 w_hold_* 寄存器用于锁存已经提出但尚未接受的请求。D-cache AXI 层还跟踪读、写响应的先后关系，确保 R 与 B 两类独立返回不会让较晚事务被误认为较早事务完成。这些改造主要服务于 FPGA 上的总线停顿、并发访问和 Linux 启动稳定性。

二、设计结果与工程优化

当前设计已通过发布包中的功能测试和性能测试，并能够借助 U-Boot 启动 Linux、进入用户交互并执行 ls 等命令。现有报告记录：性能测试时钟设为 80MHz 时，CoreMark 仿真性能分约为 3.41，实际下板测得约为 3.20。

为定位性能瓶颈，工程中加入了 BPU_PERF、PERF_MONI 以及 I/D-cache refill、evict、miss busy 等统计信号，可区分分支误预测、Cache miss、访存等待和双发射利用率。调试接口与 difftest/trace 用于核对提交 PC、寄存器写回、异常与访存行为，使性能优化建立在功能正确的基础上。

Cache 与 AXI 方向的主要工程优化包括：提前 lookup 与 MMU 地址翻译并行、D-cache 请求 skid buffer、AXI 请求保持、读响应归属记录、dirty 写回准备状态以及响应顺序控制。这些工作都能够缩短关键路径并提高随机 ready/valid 停顿下的可靠性。基于我们自主的乘法器 ip 核启动 linux 串口内容如图 2 所示：

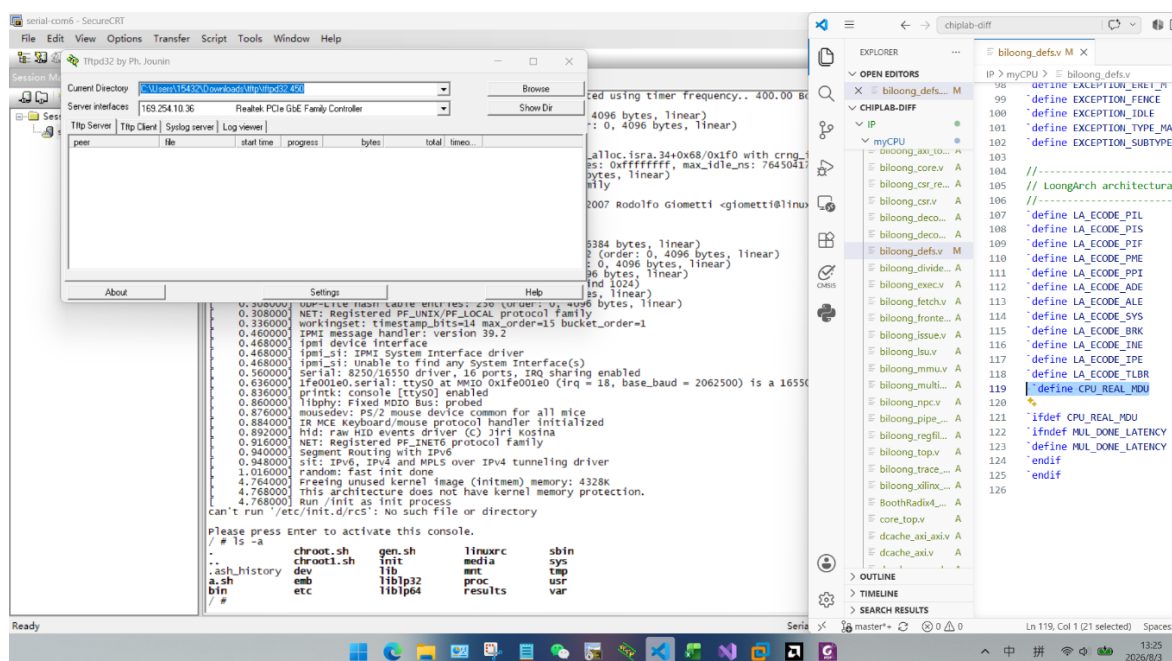


图 2 U-Boot 串口启动 linux 界面

三、参考文献

- [1] LoongsonEDU, open-LA 500: [EB/OL]. <https://gitee.com/loongson-edu/open-la500>
- [2] UltraEmbedded. biRISC-V: 32-bit RISC-V CPU with dual issue capability[EB/OL]. <https://github.com/ultraembedded/biriscv>.
- [3] 龙芯中科技术股份有限公司. 龙芯架构参考手册 卷一：基础架构[R].
- [4] ARM Limited. AMBA AXI and ACE Protocol Specification[R].
- [5] 张晨曦等. 计算机系统结构（第2版）[M]. 北京：高等教育出版社.

四、致谢

感谢董梁老师、邢高峰老师、路亮同学的悉心指导和陪伴。

2026 年 8 月